

DIP HW3_4

313512009 施丞蔚

1. Please design a highboost method including the Sobel and Laplacian filter in pp.183-195 to enhance the image, 'bodybone.bmp' as Fig. 3.49 (e). Please describe the your highboost filter, procedures, final enhanced image and print out the source code? (40)

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- parameters (feel free to tweak) -----
GAUSS_SIGMA = 1.5      # mild denoise on L
ALPHA        = 4.0      # weight for Laplacian term
BETA         = 1.0      # weight for Sobel term
LAP_SIGN     = -1       # -1 => L - ALPHA*Lap, +1 => L + ALPHA*Lap
USE_FULL_GRADIENT = False # True => (Gx+Gy)/2 ; False =>
sqrt(Gx^2+Gy^2)
# -----

def load_color(candidates):
    for p in candidates:
        if p and os.path.exists(p):
            img = cv2.imread(p, cv2.IMREAD_COLOR)
            if img is not None:
                return img, p
    raise FileNotFoundError("fish.jpg not found. Tried:\n - " + "\n - "
        ".join(candidates))

def percentile_stretch01(x, low=1.0, high=99.5, gamma=0.8):
    """For nicer visualization only (does not affect computation)."""
    x = x.astype(np.float32)
    lo, hi = np.percentile(x, low), np.percentile(x, high)
    y = np.clip((x - lo) / (hi - lo + 1e-8), 0, 1)
    return np.power(y, gamma)

def main():
    # 1) Read color image
```

```

bgr, used =
load_color(["Bodybone.bmp", "./Bodybone.bmp", "/mnt/data/Bodybone.bmp", "fish.jpg",
"./fish.jpg", "/mnt/data/fish.jpg"])

# 2) Convert BGR->Lab, get L in [0,1]
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
L = lab[:, :, 0].astype(np.float32) / 255.0
a = lab[:, :, 1]
b = lab[:, :, 2]

# 3) Mild denoise on L
L_blur = cv2.GaussianBlur(L, (0, 0), GAUSS_SIGMA)

# 4) Sobel on L
gx = cv2.Sobel(L_blur, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(L_blur, cv2.CV_32F, 0, 1, ksize=3)
if USE_FULL_GRADIENT:
    sobel_term = 0.5 * (gx + gy) # can be +/- ; we'll normalize for
stability
    # scale to [0,1] for combination
    smin, smax = sobel_term.min(), sobel_term.max()
    sobel_term = (sobel_term - smin) / (smax - smin + 1e-8)
else:
    sobel_mag = np.sqrt(gx * gx + gy * gy)
    sobel_term = sobel_mag / (sobel_mag.max() + 1e-8) # [0,1]

# 5) Laplacian on L (second derivative, +/-)
lap = cv2.Laplacian(L_blur, cv2.CV_32F, ksize=3)

# 6) High-boost on L
L_enh = L + LAP_SIGN * ALPHA * lap + BETA * sobel_term
L_enh = np.clip(L_enh, 0.0, 1.0)

# 7) Merge back to color
lab_out = np.dstack([(L_enh * 255.0).astype(np.uint8), a, b])
bgr_out = cv2.cvtColor(lab_out, cv2.COLOR_Lab2BGR)

# ----- Prepare colorized intermediate displays (for Sobel &
Laplacian) -----
# Sobel display (0..1 -> 0..255 as L) + original a/b to keep green
tone
sobel_L_disp = (np.clip(percentile_stretch01(sobel_term), 0, 1) *
255).astype(np.uint8)

```

```

sobel_lab      = np.dstack([sobel_L_disp, a, b])
sobel_bgr      = cv2.cvtColor(sobel_lab, cv2.COLOR_Lab2BGR)

# Laplacian display (map to 0..1 -> 0..255 as L) + original a/b
lap_norm = (lap - lap.min()) / (lap.max() - lap.min() + 1e-8)
lap_L_disp = (np.clip(percentile_stretch01(lap_norm), 0, 1) *
255).astype(np.uint8)
lap_lab      = np.dstack([lap_L_disp, a, b])
lap_bgr      = cv2.cvtColor(lap_lab, cv2.COLOR_Lab2BGR)

# 8) Show three images
plt.figure(figsize=(10, 12))

plt.subplot(3, 2, 1)
plt.imshow(cv2.cvtColor(sobel_bgr, cv2.COLOR_BGR2RGB))
title_sobel = "The result of Sobel filter"
title_sobel += " (full gradient)" if USE_FULL_GRADIENT else " "
(magnitude) "
plt.title(title_sobel + " - colored")
plt.axis("off")

plt.subplot(3, 2, 2)
plt.imshow(cv2.cvtColor(lap_bgr, cv2.COLOR_BGR2RGB))
plt.title("The result of Laplacian filter - colored")
plt.axis("off")

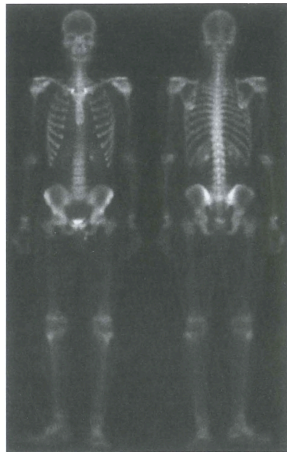
plt.subplot(3, 1, 3)
plt.imshow(cv2.cvtColor(bgr_out, cv2.COLOR_BGR2RGB))
sign_str = "+" if LAP_SIGN > 0 else "-"
grad_str = "Full_gradient" if USE_FULL_GRADIENT else "| $\nabla$ L|"
plt.title(f"Final enhanced image (color preserved)\n"
          f"L {sign_str} {abs(ALPHA)}*Lap(L) + {BETA}*{grad_str}")
plt.axis("off")

plt.tight_layout()
plt.show()

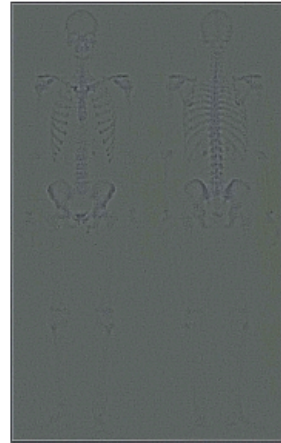
if __name__ == "__main__":
    main()

```

original image



The result of Sobel filter (magnitude) — colorized The result of Laplacian filter — colorized



Final enhanced image (color preserved)

$$L - 4.0 * \text{Lap}(L) + 1.0 * |\nabla L|$$



2. Repeat (1) steps in the image ‘fish.jpg’? (40)

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- parameters (feel free to tweak) -----
GAUSS_SIGMA = 1.5      # mild denoise on L
ALPHA        = 1.2      # weight for Laplacian term
BETA         = 0.8      # weight for Sobel term
LAP_SIGN     = -1       # -1 => L - ALPHA*Lap, +1 => L + ALPHA*Lap
USE_FULL_GRADIENT = False # True => (Gx+Gy)/2 ; False =>
sqrt(Gx^2+Gy^2)
# -----

def load_color(candidates):
    for p in candidates:
        if p and os.path.exists(p):
            img = cv2.imread(p, cv2.IMREAD_COLOR)
            if img is not None:
                return img, p
    raise FileNotFoundError("fish.jpg not found. Tried:\n - " + "\n - " +
                            ".join(candidates))

def percentile_stretch01(x, low=1.0, high=99.5, gamma=0.8):
    """For nicer visualization only (does not affect computation)."""
    x = x.astype(np.float32)
    lo, hi = np.percentile(x, low), np.percentile(x, high)
    y = np.clip((x - lo) / (hi - lo + 1e-8), 0, 1)
    return np.power(y, gamma)

def main():
    # 1) Read color image
    bgr, used = load_color(["fish.jpg", "./fish.jpg",
                            "/mnt/data/fish.jpg"])
    print(f"[INFO] Loaded: {used}, shape={bgr.shape}")

    # 2) Convert BGR->Lab, get L in [0,1]
    lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
    L = lab[:, :, 0].astype(np.float32) / 255.0
    a = lab[:, :, 1]
    b = lab[:, :, 2]
```

```

# 3) Mild denoise on L
L_blur = cv2.GaussianBlur(L, (0, 0), GAUSS_SIGMA)

# 4) Sobel on L
gx = cv2.Sobel(L_blur, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(L_blur, cv2.CV_32F, 0, 1, ksize=3)
if USE_FULL_GRADIENT:
    sobel_term = 0.5 * (gx + gy) # can be +/- ; we'll normalize for
stability
    # scale to [0,1] for combination
    smin, smax = sobel_term.min(), sobel_term.max()
    sobel_term = (sobel_term - smin) / (smax - smin + 1e-8)
else:
    sobel_mag = np.sqrt(gx * gx + gy * gy)
    sobel_term = sobel_mag / (sobel_mag.max() + 1e-8) # [0,1]

# 5) Laplacian on L (second derivative, +/-)
lap = cv2.Laplacian(L_blur, cv2.CV_32F, ksize=3)

# 6) High-boost on L
L_enh = L + LAP_SIGN * ALPHA * lap + BETA * sobel_term
L_enh = np.clip(L_enh, 0.0, 1.0)

# 7) Merge back to color
lab_out = np.dstack([(L_enh * 255.0).astype(np.uint8), a, b])
bgr_out = cv2.cvtColor(lab_out, cv2.COLOR_Lab2BGR)

# ----- Prepare colorized intermediate displays (for Sobel &
Laplacian) -----
# Sobel display (0..1 -> 0..255 as L) + original a/b to keep green
tone
sobel_L_disp = (np.clip(percentile_stretch01(sobel_term), 0, 1) *
255).astype(np.uint8)
sobel_lab = np.dstack([sobel_L_disp, a, b])
sobel_bgr = cv2.cvtColor(sobel_lab, cv2.COLOR_Lab2BGR)

# Laplacian display (map to 0..1 -> 0..255 as L) + original a/b
lap_norm = (lap - lap.min()) / (lap.max() - lap.min() + 1e-8)
lap_L_disp = (np.clip(percentile_stretch01(lap_norm), 0, 1) *
255).astype(np.uint8)
lap_lab = np.dstack([lap_L_disp, a, b])
lap_bgr = cv2.cvtColor(lap_lab, cv2.COLOR_Lab2BGR)

```

```

# 8) Show three images
plt.figure(figsize=(10, 12))

plt.subplot(3, 2, 1)
plt.imshow(cv2.cvtColor(sobel_bgr, cv2.COLOR_BGR2RGB))
title_sobel = "The result of Sobel filter"
title_sobel += " (full gradient)" if USE_FULL_GRADIENT else " "
(magnitude) "
plt.title(title_sobel + " - colored")
plt.axis("off")

plt.subplot(3, 2, 2)
plt.imshow(cv2.cvtColor(lap_bgr, cv2.COLOR_BGR2RGB))
plt.title("The result of Laplacian filter - colored")
plt.axis("off")

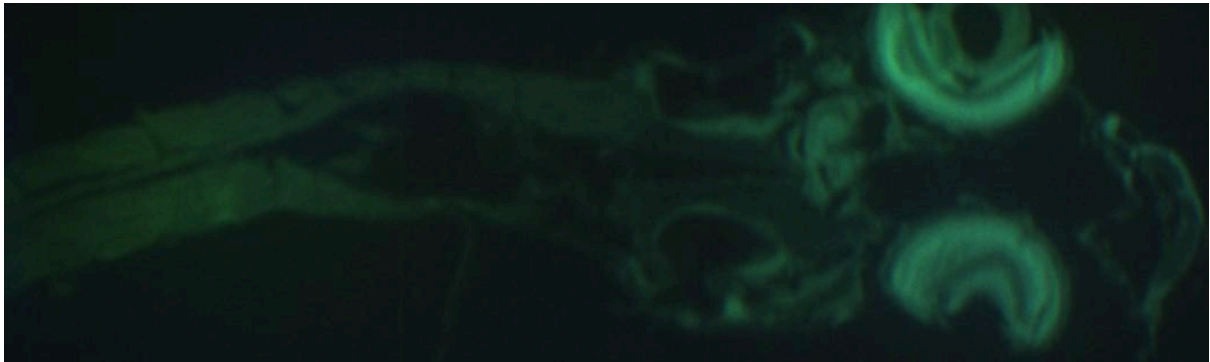
plt.subplot(3, 1, 3)
plt.imshow(cv2.cvtColor(bgr_out, cv2.COLOR_BGR2RGB))
sign_str = "+" if LAP_SIGN > 0 else "-"
grad_str = "Full_gradient" if USE_FULL_GRADIENT else " $|\nabla L|$ "
plt.title(f"Final enhanced image (color preserved)\n"
          f"L {sign_str} {abs(ALPHA)}*Lap(L) + {BETA}*{grad_str}")
plt.axis("off")

plt.tight_layout()
plt.show()

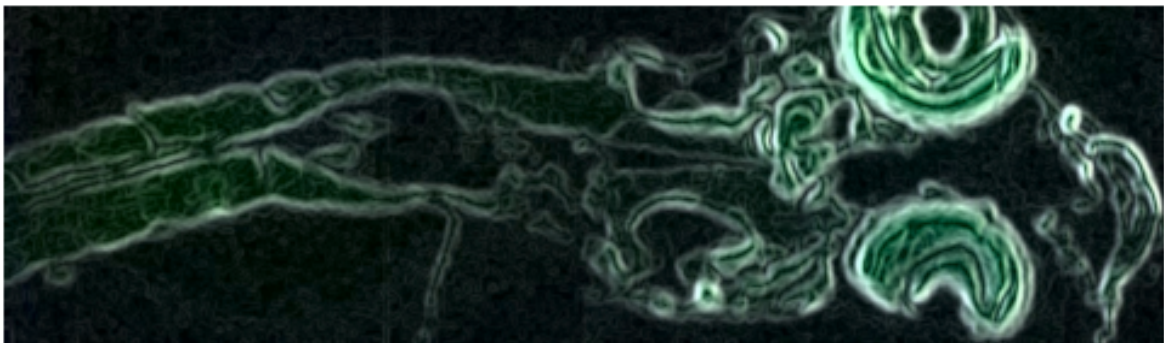
if __name__ == "__main__":
    main()

```

original image



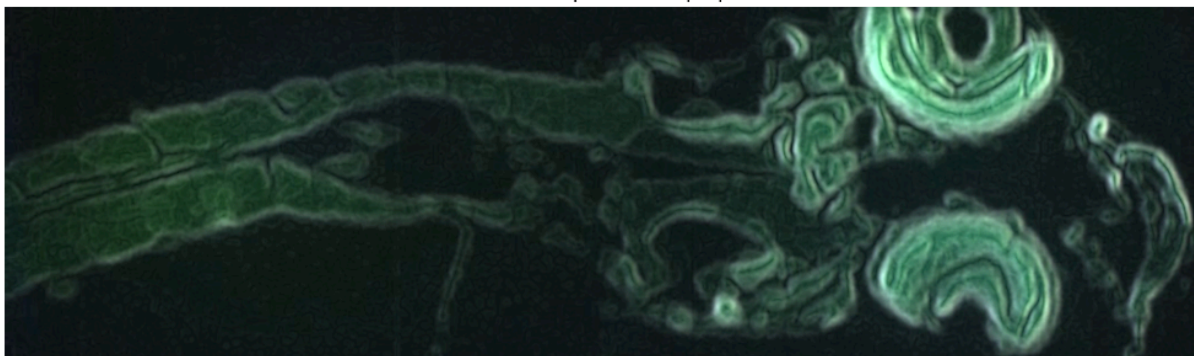
The result of Sobel filter (magnitude) — colorized



The result of Laplacian filter — colorized



Final enhanced image (color preserved)
 $L - 1.2 * \text{Lap}(L) + 0.8 * |\nabla L|$



3. Please comment and compare your two designed filters and results ?

(20)

兩張圖皆採 high-boost: 以 Sobel 提升邊緣、Laplacian 調整局部對比。對 bodybone (灰階) 我直接在亮度上運算; 對 fish (彩色) 則轉到 Lab 僅處理 L 通道, 最後合回原 a/b 色度以保留綠色。Sobel 以梯度幅值表徵邊緣; Laplacian 採經典銳化符號 $L - \alpha \nabla^2 L$ (避免正 Laplacian 造成白暈), 並在前端加小 σ 高斯抑噪。

效果上, bodybone 的肋骨、脊柱邊界更清楚, 低對比部位被拉開, 但 α 、 β 過大會放大雜訊與局部過曝; fish 的鱗片與結構線條變得利落, 因只動 L 通道幾乎不產生彩邊, 整體自然度較佳。從噪聲觀感與色彩忠實度看, Lab-L 做法更穩健: 邊緣變銳但顏色不飄移; 灰階直接處理則對雜訊較敏感。

參數上, bodybone 用較大的 α (約 3.5–4.5) 能把骨架拉出, β 約 0.8–1.0; fish 因原圖較平滑, α 約 1.2–1.6、 β 約 0.8–1.0 即可。若出現邊緣過銳或光暈, 降低 α 或加大預先平滑; 若邊緣不夠明顯, 微調 β 。